

SOFTWARE ENGINEERING LAB-3

COMPONENT MODELLING & ARCHITECTURAL PATTERN SELECTION

Centralized Audit Trail Compliance Engine

Name: Basawaraj Panchal
SRN: PES1UG24AM350
Section: F

1. Objective

To evaluate different architectural styles, select the most appropriate architecture for the Centralized Audit Trail Compliance Engine, and design a UML Component Diagram showing the major components, interfaces, dependencies, and interactions between them.

2. Scenario Overview

The system is a Centralized Audit Trail Compliance Engine that ingests raw audit and application event logs, masks personally identifiable information (PII) such as Social Security Numbers and credit card numbers before storage, and lets a Compliance Officer search historical audit records. A Security Auditor can generate tamper-evident, immutable exports of audit data with a cryptographic integrity hash, and later verify that an export has not been modified.

3. Key Requirements

Functional Requirements:

- FR-001: Parse structured JSON audit logs, identify PII fields (SSN, credit card numbers), and mask them before storing records.
- FR-002: Ingest enterprise application event logs and create corresponding audit records in the centralized audit trail.
- FR-003: Allow a Compliance Officer to query historical audit records using filtering criteria.
- FR-004: Allow a Security Auditor to generate an immutable audit export with selected records and a SHA-256 integrity hash.
- FR-005: Verify the integrity of an audit export by recalculating its hash and reporting whether it has been modified.

Non-Functional Requirements:

- NFR-001 (Performance): Support complex filtering across 10 million historical audit records in under 1 second.
- NFR-002 (Security): Ensure sensitive PII is never persisted in cleartext in audit storage.

4. Architectural Style Analysis

Architecture	Advantages	Disadvantages	Suitability
Layered Architecture	Clear separation of concerns, easy to understand and maintain	A single scaling unit forces ingestion, search and export to share resources, hurting the sub-second query target	Medium
Microservices Architecture	Independent scaling, fault isolation, independent deployment of ingestion, search and export	Operational complexity, network latency, data consistency challenges across services	High
Client-Server Architecture	Centralized control, simple deployment and data consistency	Single point of failure and scalability bottleneck — unacceptable for a	Low

		compliance system requiring high availability	
--	--	---	--

5. Architecture Selection

Selected Architecture: Microservices Architecture

We chose Microservices Architecture for the Centralized Audit Trail Compliance Engine because the system's three functional areas — log ingestion with PII masking, historical search, and secure export with integrity verification — have very different workload profiles and can be developed, scaled, and secured independently.

Reason 1 — Independent Scaling for a Strict Performance Target

NFR-001 requires filtered queries across 10 million records in under 1 second. Microservices Architecture lets the Audit Search Service be deployed on dedicated, search-optimized infrastructure and scaled independently of ingestion and export, without those workloads competing for the same resources as they would in a single monolithic layer.

Reason 2 — Isolation Between High-Throughput Ingestion and Heavy Export Processing

Log ingestion and PII masking must keep up with a continuous stream of enterprise events, while export generation and hash verification are periodic, CPU-heavy operations. Splitting these into separate services means a slow or failing export job cannot block or degrade real-time audit log ingestion.

6. Security Advantage

The PII Masking Service is isolated as its own microservice and is the only component that handles unmasked audit data. Once a record passes through masking, every other service — Search, Export, and Integrity Verification — only ever reads masked records from the Audit Data Store. This directly satisfies NFR-002 by containing all cleartext PII handling within a single, well-defined security boundary.

7. Performance Benefit

Because the Audit Search Service is a separate, independently deployable component, it can be backed by its own indexing and read-optimized storage and scaled horizontally without being constrained by the ingestion or export services. This isolation is what makes it feasible to meet NFR-001's requirement of filtering 10 million audit records in under 1 second.

8. UML Component Diagram

The component diagram below represents the six major components of the Centralized Audit Trail Compliance Engine, grouped by the epic they support, and the interfaces through which they communicate.

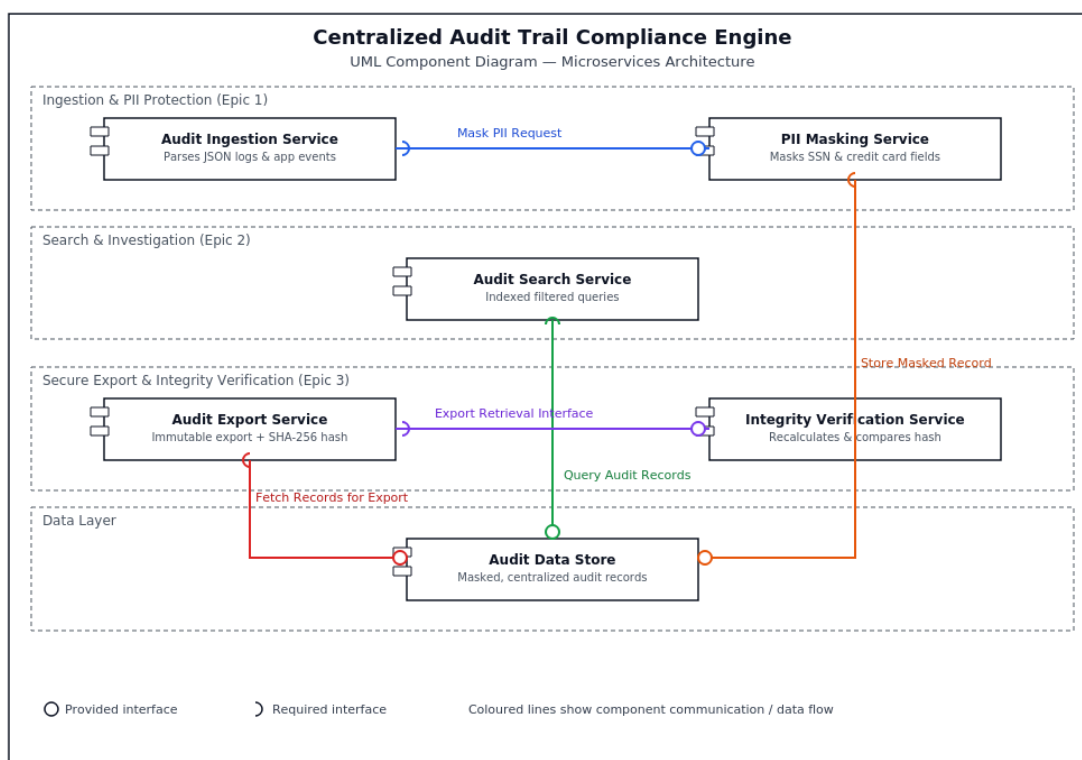


Figure 1: UML Component Diagram for the Centralized Audit Trail Compliance Engine

9. Component Interfaces and Interactions

Interface	Components	Purpose
Mask PII Request	Audit Ingestion Service ↔ PII Masking Service	Sends parsed JSON/event log records to be masked before they are persisted (FR-001)
Store Masked Record	PII Masking Service ↔ Audit Data Store	Writes the masked audit record into the centralized audit trail (FR-002, NFR-002)
Query Audit Records	Audit Search Service ↔ Audit Data Store	Runs filtered historical queries for a Compliance Officer against indexed audit data (FR-003, NFR-001)
Fetch Records for Export	Audit Export Service ↔ Audit Data Store	Retrieves the audit records selected by a Security Auditor for export (FR-004)
Export Retrieval Interface	Audit Export Service ↔ Integrity Verification Service	Supplies the generated export and its original SHA-256 hash so integrity can be recalculated and compared (FR-005)

10. Data Flow

Enterprise Systems / Raw Logs

↓

Audit Ingestion Service

↓

PII Masking Service

↓

Audit Data Store (masked record stored)

Compliance Officer

↓

Audit Search Service

↓

Audit Data Store (filtered query results returned)

Security Auditor

↓

Audit Export Service

↓

Audit Data Store (records fetched, export + SHA-256 hash generated)

↓

Integrity Verification Service (recalculates hash, confirms export validity)

11. Conclusion

Microservices Architecture is appropriate for the Centralized Audit Trail Compliance Engine because it lets ingestion, PII masking, search, export, and integrity verification be scaled, deployed, and secured independently. It supports the strict sub-second search performance target, keeps cleartext PII isolated within a single service boundary, and prevents heavy export or verification workloads from affecting real-time audit log ingestion.